

Full Stack Web Development





CHAPTER-3

Java FX



Java FX Introduction

JavaFX is a **Java library and GUI toolkit**.

- Designed for building **rich Internet applications**.
- Supports development of **desktop, web, and mobile apps**.
- Compatible with **Windows, Linux, iOS, Android**, etc.

JavaFX History

Developed by **Sun Microsystems** (later acquired by Oracle).

Introduced to support **Rich Internet Applications (RIA)**.

Replaced Java Swing for more modern GUI development.



Java FX Introduction conti.

What are RIAs?

Rich Internet Applications:

Web apps with **desktop-like functionality**.

Provide **better user experience** than standard web apps.

Java FX and Swing

Feature	Java Swing	JavaFX
UI Components	OS-dependent	OS-independent
Media Support	Limited	Built-in support
Animation Support	Basic	Advanced
Performance	Moderate	Hardware-accelerated



Java FX Introduction Conti.

Cross-Platform Versatility

JavaFX runs on:

- Desktops
- Web browsers
- Smartphones (iOS & Android)
- TVs and Tablets
- Enables “Write Once, Run Anywhere”.

Why JavaFX Was Needed

Previously:

UI, media, and effects required multiple libraries.

JavaFX Solution:

Unified library for all features.
Simplified client-side application development.





History of Java FX

History of JavaFX

JavaFX was formerly developed by Chris Oliver. At that time, he was serving for a company named See Beyond Technology Corporation. Initially, the JavaFX project was recognized as Form Follows Functions (F3). This project was designed with the aim of providing richer interfaces for developing GUI applications. Later in June 2005, Sun Micro-systems took the F3 project and changed its name from F3 to JavaFX.

Timeline of JavaFX

2005 - Sun Microsystems took over the See Beyond company in June 2005 and acquired the F3 project as JavaFX.

2007 - JavaFX was officially declared at Java One, a worldwide web conference that is held yearly.

2008 - Net Beans integration with JavaFX was made open. The Java Standard Development Kit for JavaFX 1.0 was also released in the same year.



History of Java FX conti.

2009 - The next version of JavaFX was released, i.e., JavaFX 1.2, and the support for JavaFX Mobile was also introduced. In the same year only, Oracle Corporation also acquired Sun Microsystems.

2010 - JavaFX version 1.3 was released in 2010.

2011 - In 2011, JavaFX version 2.0 came out.

2012 - The support for JavaFX Mac OS for desktop was introduced.

2014 - The most advanced version of JavaFX, i.e., JavaFX 8, was released as an indispensable part of Java on the 18th of March 2014.





Features of Java FX

JavaFX is an open-source framework based on Java, used for advancing rich client applications. JavaFX is recognized as the replacement or successor of the Java Swing in the field of graphical user interface (GUI) development technology in the platform of Java. The JavaFX library is available as a public Java application programming interface (API).

The JavaFX library comprises numerous peculiarities that make it a handpicked option for developers to develop rich client applications. These features are as follows:

1. **JavaFX as a Java Library**

Full Java support (Multithreading, Lambda, etc.)

Integrated with Java ecosystem

Develop in any standard IDE

Access to all Java tools and frameworks





Features of Java FX

2. Platform Independence

JavaFX runs on any platform with **JVM**

Supports multiple languages:

Java, Groovy, Scala, JRuby

"Write Once, Run Anywhere" with JavaFX

4. Graphics: Prism Engine

Hardware-accelerated graphics pipeline

Prism Engine for high-performance rendering

Fallback to **software rendering** if no GPU

Smooth graphics across devices

3. FXML & Scene Builder

FXML: XML-based markup language for UI

Scene Builder: Visual editor for FXML

Drag-and-drop UI design

Easy integration with Java code in IDEs

5. WebView Component

Embeds web pages using **WebKitHTML**

Supports **JavaScript-Java API** interaction

Adds web capabilities within desktop apps

Enables hybrid application design



Features of Java FX

6. Built-in UI Controls

Rich set of **OS-independent UI components**

Buttons, sliders, tables, etc.

No dependency on native widgets

Ideal for cross-platform design

8. Rich Set of APIs

APIs for:

2D/3D graphics

Media

Data models (Lists, Maps)

Full access to Java features:

Generics, Annotations, Lambda,
Multithreading

7. CSS Styling Support

Style UI using **CSS**

Customize look-and-feel with ease

Allows separation of UI logic and design

Familiar styling approach for web
developers

9. High-Performance Media Engine

Media engine based on **GStreamer**

Supports low-latency audio/video
playback

Stable multimedia integration

Plays major formats in high quality



Architecture of Java FX

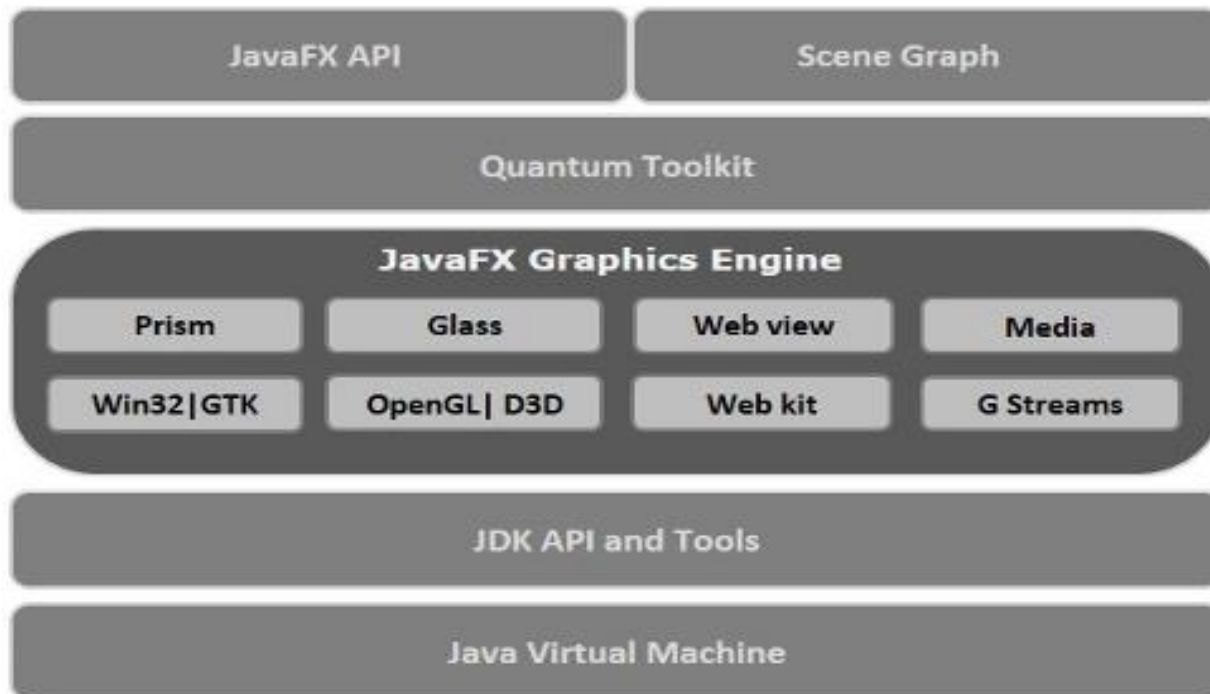
Architecture of JavaFX

JavaFX has numerous built-in elements that are interconnected with each other. JavaFX library comprises a valuable collection of APIs, classes, and interfaces that are more than sufficient to produce rich internet applications and GUI applications with intense graphics that can run consistently over multiple platforms.



Architecture conti.

The subsequent figure displays the complete architecture of the JavaFX platform. Here you can examine the elements that support JavaFX API.



Architecture conti.

1. **JavaFX API** - The topmost layer of JavaFX architecture holds a JavaFX public API that implements all the required classes that are capable of producing a full-featured JavaFX application with rich graphics. The list of all the important packages of this API is as follows.
 - **javafx.animation**: It includes classes that are used to combine transition-based animations such as fill, fade, rotate, scale and translation, to the JavaFX nodes (collection of nodes makes a scene graph).
 - **javafx.css** – It comprises classes that are used to append CSS–like styling to the JavaFX GUI applications.
 - **javafx.geometry** – It contains classes that are used to represent 2D figures and execute methods on them.
 - **javafx.scene** – This package of JavaFX API implements classes and interfaces to establish the scene graph. In extension, it also renders sub-packages such as canvas, chart, control, effect, image, input, layout, media, paint, shape, text, transform, web, etc. These are the diverse elements that sustain this precious API of JavaFX.
 - **javafx.application** – This package includes a collection of classes that are responsible for the life cycle of the JavaFX application.

Architecture conti.

- **javafx.event** – It includes classes and interfaces that are used to perform and manage JavaFX events.
 - **javafx.stage** – This package of JavaFX API accommodates the top-level container classes used for the JavaFX application.
- 2. Scene Graph** - A Scene Graph is the starting point of the development of any of the GUI Applications. In JavaFX, all the GUI Applications are made using a Scene Graph only. The Scene Graph includes the primitives of the rich internet applications that are known as nodes. In simple words, a single component in a scene graph is known as a node. In general, a scene graph is made up of a collection of nodes. All these nodes are organized in the form of a hierarchical tree that describes all of the visual components of the application's user interface (UI). A node instance can be appended to a scene graph only once. The nodes of a scene graph can have numerous segments like Effects, Opacity, Transforms, Event Handlers, Application Specific States. The nodes are of three general types.

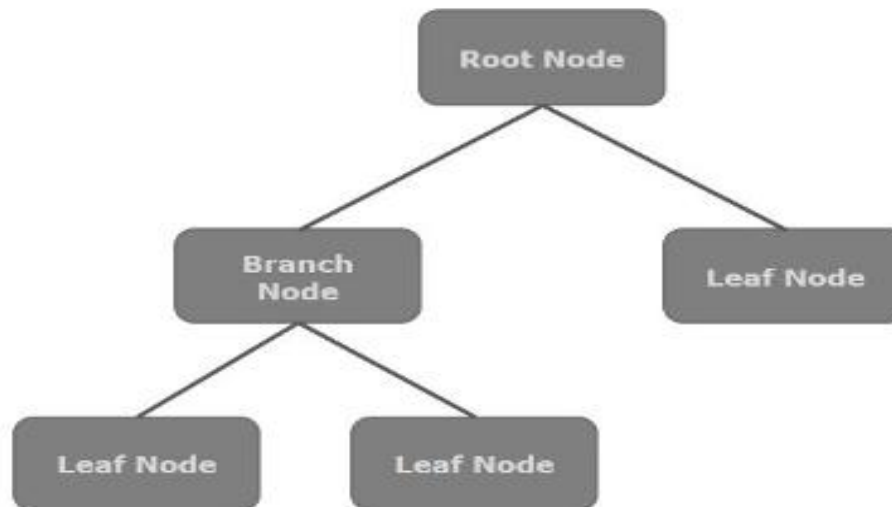
Architecture conti.

These are as follows:

Root Node - A root node is a node that does not have any node as its parent.

Leaf Node - A leaf node is a node that does not contain any node as its children.

Branch Node - A branch node is a node that contains a node as its parent and also has a node as its children.



Architecture conti.

3. Quantum Toolkit - Quantum Toolkit is used to connect prism and glass windowing tool kits collectively and makes them prepared for the above layers in the stack. In simple words, it ties Prism and GWT together and makes them available to JavaFX.

GRAPHIC ENGINE

1. Prism - The graphics of the JavaFX applications are based on the hardware-accelerated graphics rendering pipeline, commonly known as Prism. The Prism engine supports smooth JavaFX graphics that can be executed swiftly when utilized with a backed graphics card or graphics processing unit (GPU). In the situation where the system does not contain the graphic cards, then the prism engine defaults to the software rendering stack. To interpret graphics, a Prism practice – DirectX 9 on Windows XP and Vista.

DirectX 11 on Windows 7.

OpenGL on Mac and Linux, Embedded Systems.

2. Glass Windowing Toolkit - Glass Windowing Toolkit or simply Glass is a platform-dependent layer that assists in connecting the JavaFX platform to the primary operating system (OS). Glass Windowing Toolkit is very useful as it provides services such as controlling the windows, events, timers, and surfaces to the native operating system.

Architecture conti.

3. WebView - JavaFX applications can also insert web pages. To embed web pages, Web View of JavaFX uses a new HTML rendering engine technology known as WebKitHTML. WebView is used to make it possible to insert web pages within a JavaFX application. JavaScript appearing in WebView can call Java APIs and vice-versa. This element promotes different web technologies like HTML5, CSS, JavaScript, DOM, and SVG. Using web view, we can execute the HTML content from the JavaFX application and can also implement some CSS styles to the user interface (UI) part of the application.

4. Media Engine - Like the graphics pipeline, JavaFX also possesses a media pipeline that advances stable internet multimedia playback at low latency. This high-performance media engine or media pipeline is based on a multimedia framework known as Gstreamer. By applying the Media engine, the JavaFX application can support the playback of audio and video media files. The package **javafx.scene.media** covers all the classes and interfaces that can provide media functionalities to JavaFX applications.

The foregoing were the components that constitute the architecture of JavaFX.

Life Cycle of a JavaFX Application

There are in total three life cycle methods of a JavaFX Application class. These methods are -

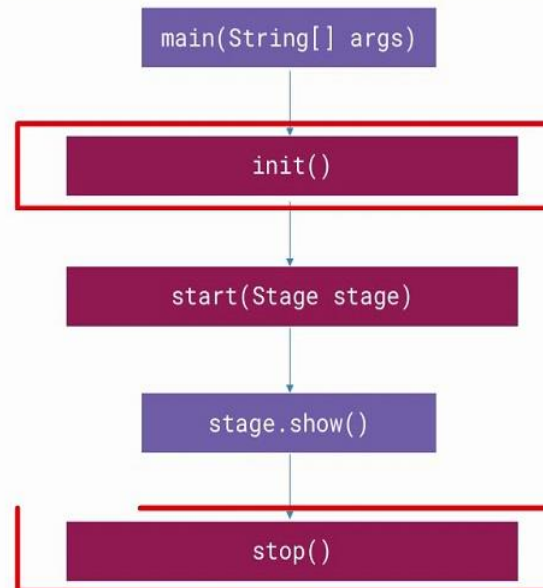
start() - The start() method is the entry point method of the JavaFX application where all the graphics code of JavaFX is to be written.

init() - The init() method is an empty method that can be overridden. In this method, the user cannot create a stage or a scene.

stop() - The stop() method is an empty method that can also be overridden, just like the init() method. In this method, the user can write the code to halt the application.

Application conti.

Lifecycle of JavaFX Application



Application conti.

Other than these methods, the JavaFX application also implements a static method known as **launch()**. This launch() method is used to launch the JavaFX application. As stated earlier, the launch() method is static, the user should call it from a static method only. Generally, that static method, which calls the launch() method, is the main() method only.

Whenever a user launches a JavaFX application, there are few actions that are carried out in a particular manner only. The following is the order given in which a JavaFX application is launched. Firstly, an instance of the application class is created.

After that, the init() method is called.

After the init() method, the start() method is called.

After calling the start() method, the launcher waits for the JavaFX application to end and then calls the stop() method.

Terminating the JavaFX application

As soon as the last window of the JavaFX application is closed, the JavaFX application is stopped implicitly. The user can turn off this function by passing the Boolean value “False” to the static method **setImplicitExit()**. This method should always be called from a static context only.

The user can also stop a JavaFX application explicitly by practicing any of the two methods, **Platform.exit()** or **System.exit(int)**.

JavaFX 2D Shapes

JavaFX 2D Shapes

Any geometrical shape that can be represented on the coordinate system using two planes, which are X and Y planes, is known as a two-dimensional shape or a 2D shape. Examples of 2D shapes are line, square, rectangle, circle, ellipse, and many more.

JavaFX gives the flexibility to the user to add and create these 2D shapes in their JavaFX applications. There are several classes in JavaFX API that are used to execute 2D shapes in the JavaFX application. All these classes of 2D shapes are part of the **javafx.scene.shape** package. The class named **Shape** is the root class of all the 2-Dimensional shapes in JavaFX. There are many classes in this package that contains various sorts of 2D shapes.

Steps to create a 2D shape are as follows:



Steps to create 2D shapes

Step 1:

Create the shape you want to add. JavaFX provides predefined classes for 2D shapes under `javafx.scene.shape`. Example: To draw a line, instantiate the `Line` class.

```
Line line = new Line();
```

Step 2:

Set Shape Properties

Objective: Define the shape's attributes using setter methods.

Set the coordinates for the **starting and ending points** of the line:

```
line.setStartX(0);  
line.setStartY(0);  
line.setEndX(100);  
line.setEndY(200);
```

Step 3:

Add the shape to a `Group` (container for visual elements):

```
Group root = new Group();  
root.getChildren().add(line);
```



Types of 2D shapes

In the **javafx.scene.shape** package of JavaFX, there are numerous shapes available which the user can use in their application. The following is the list of various classes of the 2D shapes that are provided by JavaFX.

Line - A geometrical structure that joins two coordinates (X and Y) on a 2D coordinate system is known as a Line. If a user wants to create a line in their JavaFX application, then they should instantiate the **javafx.scene.shape.Line** class.

Rectangle - A Rectangle is a geometrical shape that has two sets of two identical sides and four right angles at their joint. If a user wants to create a rectangle in their JavaFX application, then they should instantiate the **javafx.scene.shape.Rectangle** class.

Ellipse - In general, an ellipse can be defined as a curve with two focal points. In an ellipse, the sum of the distances to the focal points is constant from each point of the ellipse. If a user wants to create an ellipse in their JavaFX application, then they should instantiate the **javafx.scene.shape.Ellipse** class.

Types of 2D shapes conti.

Circle - In general, a specific sort of Ellipse that has both of its focal points at the same location is known as a circle. If a user wants to create a circle in their JavaFX application, then they should instantiate the **`javafx.scene.shape.Circle`** class.

Arc - An arc can be defined as the portion of the circumference of the circle or an ellipse. If a user wants to create an arc in their JavaFX application, then they should instantiate the **`javafx.scene.shape.Arc`** class.

Polygon - In general, a Polygon is a geometrical shape that can be formed by joining the various Co-planner line segments. If a user wants to create a polygon in their JavaFX application, then they should instantiate the **`javafx.scene.shape.Polygon`** class.

Cubic Curve - A curve of degree 3 in the XY plane is known as a cubic curve. If a user wants to create a cubic curve in their JavaFX application, then they should instantiate the **`javafx.scene.shape.CubicCurve`** class.

Quad Curve - A curve of degree 2 in the XY plane is known as a quad curve. If a user wants to create a quad curve in their JavaFX application, then they should instantiate the **`javafx.scene.shape.QuadCurve`** class.

JavaFX Text

JavaFX Texts

A user can also create a node for applying different texts in JavaFX. The text node is expressed by the class named **Text**, which belongs to the **javafx.scene.text** package. This Text class includes numerous characteristics to generate text in JavaFX and alter its appearance. Another feature of the text class is that this class also inherits the Shape class of JavaFX which belongs to the **javafx.scene.shape** package. Hence, in extension to the characteristics of the text like font, alignment, line spacing, text, etc., it also acquires the basic shape node properties such as strokeFill, stroke, strokeWidth, strokeType, etc.

To create a text node, the user needs to follow certain steps as follows:

Step 1: The first step to create a text node in a JavaFX application is to instantiate the corresponding class of the text node.

Illustration: The class Text of the package javafx.scene.text denotes the text node in JavaFX. A user can create a text by instantiating this class as given below as follows:

```
Text txt = new Text();
```


JavaFX Text Conti.

Step 2: After instantiating the corresponding class of the text node, the second step to create a text node in JavaFX is to set the properties of the required text using the setter functions.

Illustration: After instantiating the Text class, the user needs to set the value to this property using the setText() method as shown below as follows:

```
String text = "Hello how are you"
```

```
txt.setText(text);
```

```
txt.setX(50);
```

```
txt.setY(50);
```

Step 3: After doing the above two steps, the third step for the user is to add the object instantiated of the required text to the group by declaring it as a parameter of the constructor.

Illustration: In order to get a text in the JavaFX application, the user should add the instantiated object of the text node to the group.

```
Group root = new Group();
```

```
root.getChildren().add(txt);
```



Properties of JavaFX Texts

The attributes of the JavaFX Text are represented in the table below.

Attribute	Description	Setter Method
Font	It represents the font of the text.	setFont(Font value)
Line spacing	It represents the vertical space between the lines in pixels. The type of this property is double.	setLineSpacing(double spacing)
Text	It represents the text string that is to be presented. The type of this property is a string.	setText(String value)
Underline	Using this property, a user can underline the text by setting it to true. The type of this property is boolean.	setUnderLine(boolean value)
X	It represents the X coordinate of the text.	setX(double value)
Y	It represents the Y coordinate of the text.	setY(double value)
Strike through	With the help of this property, a user can put a line through the text by setting this property to true. The type of this property is boolean.	setStrikeThrough(boolean value)
Wrapping width	It represents the vertical space between the lines in pixels. The type of this property is double.	setWrappingWidth(double value)
Text alignment	It sets the text in Horizontal alignment	setTextAlignment(TextAlignment value)

JavaFX Effects

Any behavior or movement that improves the presentation of the graphics is known as an Effect. In JavaFX, we can say that any algorithm that is applied to nodes to enhance their appearance visually is an effect. In order to specify the effect, the effect property of the Node class is used.

JavaFX provides multiple ready-to-use effects in `javafx.scene.effect` package.

The base class for all effects is `Effect`.

In order to add an effect to a Node, the user needs to sequentially follow these steps as follows:

Step 1: The first step to applying an effect to a Node is to create all the nodes in a JavaFX application by instantiating their individual classes. For illustration purposes consider if a user wants to apply the glow effect to an image in their application, then they should create an image node first by instantiating the `Image` class. After that, they should set its view just as shown below as follows:

// Creating an image

```
Image img = new Image("https://media.geeksforgeeks.org/wp-content/uploads/20210224040124/JSBinCollaborativeJavaScriptDebugging6.png");
```

// Setting the image view

```
ImageView imgView = new ImageView(img);
```

// Setting the position of the image

```
imgView.setX(100);
```

```
imgView.setY(100);
```



PU



JavaFX effects conti.

// Setting the fit height and width of the image view

```
imageView.setFitHeight(200);
```

```
imageView.setFitWidth(400);
```

// Setting the preserve ratio of the image view

```
imageView.setPreserveRatio(true);
```

Step 2: After creating the image node and setting the image view, the user should instantiate the class representing the required effect that is needed to be applied to the node created. For illustration purposes of considering if the user wants to apply the glow effect in their application, then they need to instantiate the Glow class as shown below as follows:

```
Glow glow = new Glow();
```

Step 3: After instantiating the class of the required effect, the user needs to set the attributes for the instantiated effect using its setter methods. For illustration purposes consider as if the user can set the attributed of the Glow effect using the below-shown method as follows:

// Setting the level property

```
glow.setLevel(0.9);
```

Step 4: After the above three steps, the user can finally apply the required effect to the node using the `setEffect()` method.

For example - The user can set the glow effect to the image node by passing the object of the Glow class to this method as follows -

```
imageView.setEffect(glow);
```



Major JavaFX effects

Effect	Description
Glow	Brightens highlights in the node
Bloom	Produces glowing effect in bright areas
Shadow	Adds a soft shadow outside the node
InnerShadow	Adds a shadow inside the edges of the node
Reflection	Reflects the node's image below it
ColorAdjust	Adjusts hue, brightness, contrast, and saturation
SepiaTone	Applies a reddish-brown vintage tone
Blend	Mixes multiple visual inputs based on a blend mode
Lighting	Simulates light from a source for a 3D look

JavaFX Transformations

Transformation implies modifying some graphics into something different by implementing various rules. In JavaFX there are different sorts of transformations such as Translation, Scaling Up or Down, Rotation, Shearing, etc. All these transformations are expressed by several classes and these classes belong to the package **javafx.scene.transform**.

Different classes of JavaFX Transformations

In JavaFX, there are basically 4 predefined JavaFX transformations classes that are available for the user to use in their application. The following is a brief introduction of various transformations that are provided by JavaFX.

Rotation - Rotation is used to rotate the object from its source by a certain angle. The class **javafx.scene.transform.Rotate** represents rotation.

Translation - Translation is used to modify the location of the node. The class **javafx.scene.transform.Translate** represents the translation.

Scaling - Scaling is used to modify the size of the node. The class **javafx.scene.transform.Scale** represents Scaling.

Shearing - Shearing is used to altering the slope of the object in a particular direction. The class **javafx.scene.transform.Shear** represents Shearing.

Steps to apply transformations to the target node are as follows:

To apply a special transformation to a target node, a user should follow the below-given steps:

1. The first step is to instantiate the class of the respective transformation which the user wants to apply.

For example - If a user wants to apply a translate transformation on a specific node, then he should instantiate the translate class.

```
TranslateTransition translate = new TranslateTransition();
```

2. After instantiating the class, the next step is to set of properties of the instantiated class.

For example - user has instantiated the translate transformation class of JavaFX, then he should set the properties such as translate byX, translate byY, duration, and cycle count of the translated class.

```
translate.setByX(400);
```

```
translate.setDuration(Duration.millis(1000));
```

```
translate.setCycleCount(500);
```

3. Set the transformation:

For Example: Let us assume that the user wants to translate a specific node then after instantiating a transformation class and setting its value, the user should set the node using the name of the node that should play the translation.

```
translate.setNode(name of the node);
```

```
translate.play();
```

JavaFX Animations

The transition which produces the myth of motion for an object is known as animation. Animations can also be defined as the set of transformations implemented on an object over the specified duration sequentially so that the object can be displayed as it is in action. This can be achieved by the rapid display of frames. In JavaFX, the package **javafx.animation** includes all the classes that assist in applying the animations onto the nodes. All the classes of this package extend the class **javafx.animation.Animation**. JavaFX provides the classes for the transitions like RotateTransition, ScaleTransition, TranslateTransition, FadeTransition, FillTransition, StrokeTransition, etc.

Steps to apply animations to the target node are as follows:

To apply a special animation to a target node, a user should follow the below-given steps as follows:

Step 1: The first step to applying an animation to a specific node is to create that node using the respective class of the node.

```
Square sqr = new Square(100,100,100,100);  
sqr.setFill(Color.GREEN);
```

Step 2: After creating the node, the next step is to instantiate the respective animation class that is to be applied to the created node.

```
RotateTransition rotate = new RotateTransition();
```

Step 3: After instantiating the respective animation class, in the step, the user should set the properties of the animation.

```
rotate.setDuration(Duration.millis(2000));  
rotate.setAxis(Rotate.Y_Axis);  
rotate.setCycleCount(1000);
```

Step 4: In the next step, the user should set the target node on which the animation will be applied.

```
rotate.setNode(sqr);
```

Step 5: After following all the above steps, the last step to apply an animation to a target node is to play the applied animation using the play() method of the Animation class.

```
rotate.play();
```



Different classes of JavaFX animations/transitions

Transition	Description
ScaleTransition	Animates scale (zoom in/out)
TranslateTransition	Moves node from one location to another
RotateTransition	Rotates node around X, Y, or Z axis
FadeTransition	Animates node's opacity (fade in/out)
StrokeTransition	Animates stroke color between two values
FillTransition	Animates fill color between two color values
PathTransition	Moves node along a specified geometric path
ParallelTransition	Runs multiple transitions on a node at the same time

JavaFX 3D Shapes

Any solid geometrical figure that can be represented on the coordinate system using three planes, which are X, Y, and Z planes, is known as a three-dimensional shape or a 3D shape.

There are many basic 3D shapes which we see on a daily basis such as cylinders, spheres, boxes, cubes, pyramids, etc. Nevertheless, JavaFX implements only 3 classes to create 3D shapes. These classes include spheres, cylinders, and boxes. These classes are defined in the **javafx.scene.shape** package. This package provides all the methods to deal with the 3D shapes. The class Shape3D of the package javafx.scene.shape is the base class of all the 3D shape classes in javafx.

Types of 3D shapes in JavaFX

In JavaFX, 3D shapes are characterized in two separate types. These are:

1. Predefined 3D shapes: Predefined 3D shapes are the shapes whose classes are already present in the JavaFX API. These shapes are Cylinder, Sphere, and Box. In order to create these shapes in the JavaFX application, the user just needs to instantiate these classes. These classes include several properties and methods that are needed to be used in order to generate the proper shapes.

2. User Defined 3D shapes: JavaFX provides the class **javafx.scene.shape.TriangleMesh** which extends the abstract class **javafx.scene.shape.Mesh**. This class facilitates the user to define their own points, texture coordinates, and faces as the properties of the class.

steps to generate proper shapes in their JavaFX application:

1. The first step is to instantiate the class of the respective 3D shape which the user wants to build.

For Example - If a user wishes to create a Box, then he should instantiate the Box class of the JavaFX.

```
Box box = new Box();
```

2. After instantiating the class, the next step is to set of properties of the instantiated class.

For Example - If a user has instantiated the Box class of JavaFX, then he should set the properties such as Height, width, and depth of the Box class.

```
box.setHeight(100.0);
```

```
box.setDepth(80.0);
```

```
box.setWidth(60.0);
```

3. After the above two steps, the user should set the camera for the scene. In simple words, the user can set the camera to a particular position for the camera view. However, this step is optional. A user can set the camera view using the following code -

```
PerspectiveCamera cam = new PerspectiveCamera();
```

```
cam.setTranslateX(100.0);
```

```
cam.setTranslateY(80.0);
```

```
cam.setTranslateZ(-50.0);
```

```
scene.setCamera(cam)
```

steps to generate proper shapes in their JavaFX application:

4. The last step to creating a 3D shape for the JavaFX application is to attach the 3D shape to the Scene graph and set the relevant properties for the Scene and stage.

For example: If a user has instantiated the box class, then he should link that class with the Scene graph and the stage.

```
Group root = new Group();  
root.getChildren().add(box);  
Scene scene = new Scene(root,500,500);  
primaryStage.setScene(scene);  
primaryStage.setTitle("3D shape Example");  
primaryStage.show();
```

Different classes of JavaFX 3D shapes

Shape	Description	Class
Box	A cuboid defined by width, height, depth	javafx.scene.shape.Box
Sphere	A solid round shape defined by radius	javafx.scene.shape.Sphere
Cylinder	A circular shape with height and radius	javafx.scene.shape.Cylinder

JavaFX Layout

JavaFX Layouts

Layouts are the top-level container classes that describe the UI styles for objects of the scene graph. The JavaFX layout can be seen as the father node to all the separate nodes. JavaFX presents several layout panes that promote various styles of layouts.

In JavaFX, Layout describes the process in which the elements are to be viewed on the screen. It primarily establishes the scene-graph nodes. There are various built-in layout panes in JavaFX that are HBox, VBox, StackPane, FlowBox, AnchorPane, etc. Each Built-in layout is represented by a separate class that requires to be instantiated in order to implement that specific layout pane. All these classes belong to the **javafx.scene.layout** package. The **javafx.scene.layout.Pane** class is the root class for all the built-in classes of JavaFX layouts.

Steps to create a JavaFX layout

In order to create the layouts, we need to follow the following steps -

1. The first step to creating a layout in JavaFX application is that the user should instantiate the respective layout class,

For example - If a user wants to create an HBox layout, then he should instantiate the HBox Class of JavaFX layout.

```
HBox root = new HBox();
```



Steps to create a JavaFX layout conti.

2. After instantiating the respective class, the next step is to set the properties for the layout.

```
root.setSpacing(20);
```

3. After the above two steps, the last step of creating a layout in the JavaFX application is to add nodes to the layout object.

```
root.getChildren().addAll(<NodeObjects>);
```

Different classes of JavaFx Layout

Layout	Description
HBox	Arranges nodes in a horizontal row
VBox	Arranges nodes in a vertical column
GridPane	Places nodes in a grid of rows and columns — ideal for forms
BorderPane	Divides the scene into Top, Bottom, Left, Right, Center
StackPane	Stacks nodes on top of each other (like layers)
FlowPane	Wraps nodes horizontally or vertically based on container space

JavaFX UI

JavaFX UI Controls

The UI controls are the elements that are truly exposed to the user for intercommunication or information exchange. A layout represents the structure of the UI elements on the screen. The response is the reaction of the UI component when some event has happened on it. However, the package **javafx.scene.control** implements all the required classes for the UI elements like Button, Label, etc. Every class describes a particular UI control and explains few methods for their styling.



Different classes of JavaFX UI controls

Control	Description	Package Name
Button	Triggers an action when clicked	javafx.scene.control.Button
Label	Displays non-editable text	javafx.scene.control.Label
TextField	Accepts single-line user input in text format	javafx.scene.control.TextField
PasswordField	Accepts password input (characters are hidden)	javafx.scene.control.PasswordField
CheckBox	Allows multiple true/false selections	javafx.scene.control.CheckBox
RadioButton	Allows one selection among a group of options	javafx.scene.control.RadioButton
Slider	Provides a sliding scale for selecting numeric values	javafx.scene.control.Slider
ProgressBar	Displays progress of a task or operation visually	javafx.scene.control.ProgressBar
Menu	Creates dropdown menus or menu bars in applications	javafx.scene.control.Menu
ScrollBar	Adds scrolling capability (vertical/horizontal) to content	javafx.scene.control.ScrollBar



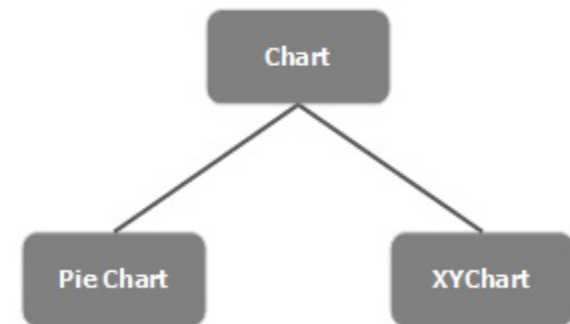
JavaFX Chart

In general, the chart can be defined as the graph or diagram which represents the data in the form of symbols. Charts are mainly used to represent large quantities of data and the relationship between parts of the data. We can create different kinds of charts to represent different kinds of information. In JavaFX, we can create the charts by using the classes provided by the package **javafx.scene.chart**.

Types of charts

Charts in JavaFX can be classified into the subsequent types:

1. **Pie Chart** - In Pie Chart, the areas of a circle are used to define several proportions of the complete information. In JavaFX, the class **javafx.scene.chart.PieChart** is used to deal with the pie chart
2. **XYChart** - The data is sketched on the XY (horizontal and vertical) axes. The X-axis depicts one type of value while the Y-axis depicts the other type of value. The mapping is done between the values plotted on X and Y charts to display the appropriate information. In JavaFX, the class **javafx.scene.chart.XYChart** is practiced to deal with the XYChart.



Different classes of JavaFX Charts

In JavaFX, there are many charts that are used by the users in their applications. Below is the list of usually used charts while the GUI is designed using JavaFX.

Chart Type	Description	JavaFX Class
Pie Chart	Divides a circle into proportional segments to represent parts of a whole.	<code>javafx.scene.chart.PieChart</code>
Bar Chart	Displays data using rectangular bars whose lengths are proportional to the values. Supports vertical/horizontal.	<code>javafx.scene.chart.BarChart</code>
Line Chart	Shows data as a series of points connected by lines, useful for trends over time.	<code>javafx.scene.chart.LineChart</code>
Area Chart	Similar to a line chart but with the area between the line and axis filled, highlighting quantitative differences.	<code>javafx.scene.chart.AreaChart</code>

Java FX Event Handling

JavaFX Event Handling

JavaFX grants users the adaptability to generate numerous types of applications such as Desktop Applications, Web applications, and graphical applications. In recent-day applications, the users perform an essential role in the precise execution of the application. The user needs to interact with the application in most cases.

In JavaFX, An event occurs whenever the user interacts with the application nodes. There are multiple references by using which, the user can create the event. For example, the user can make use of a mouse or it can press any button on the keyboard or it can scroll any page of the application in order to generate an event. Hence, we can say that the events are fundamentally the announcements that tell us that something has happened at the user's end. A perfect Application is the one that takes the least amount of time in handling the events.

Java FX Event Handling conti.

Processing Events in JavaFX

In JavaFX, events are primarily used to inform the application regarding the actions chosen by the user. JavaFX implements the tool to achieve the events, route the event to its target, and granting the application to handle the events.

JavaFX presents the class **javafx.event.Event** which includes all the subclasses describing the varieties of events that can be created in JavaFX. Any event is an instance of the class Event or any of its subclasses.

There are several events in JavaFX i.e. MouseEvent, KeyEvent, ScrollEvent, DragEvent, etc. A user can further specify their personal event by inheriting the class **javafx.event.Event**.

Types of Event:

Event Type	Description
Foreground Events	Events triggered by direct user interaction with the GUI (e.g., click, scroll).
Background Events	Events triggered by system-level operations or internal processes.



Java FX Event Handling conti.

Different class of of JavaFX Event Handling

Event Type	Description	JavaFX Class
KeyEvent	Represents keyboard interactions such as key pressed, key released, and key typed.	<code>javafx.scene.input.KeyEvent</code>
MouseEvent	Represents mouse actions like click, press, release, movement, and entering/exiting a target area.	<code>javafx.scene.input.MouseEvent</code>
WindowEvent	Represents window-based actions such as showing, hiding, opening, or closing a window.	<code>javafx.stage.WindowEvent</code>
DragEvent	Represents drag-and-drop interactions, including drag entered, drag over, drag dropped, and exiting target.	<code>javafx.scene.input.DragEvent</code>

× ○ DIGITAL LEARNING CONTENT



Parul[®] University



www.paruluniversity.ac.in

